

NIS4307 人工智能导论 (A 类)

作业报告

任务名称 基于 BERTweet、RAG 与大语言模型的
可解释谣言检测系统

完成组号 4

小组人员 陈忠淳 沈润泽 戴铭辰 谢紫浩

完成时间 2026 年 6 月 4 日——2026 年 6 月 14 日

目录

1 任务目标	3
2 系统设计与实现	3
2.1 实施方案	3
2.2 核心代码分析	3
2.2.1 数据处理模块——src/data.py	4
2.2.2 模型定义与推理——src/model.py、src/predict.py	4
2.2.3 训练流程——src/train.py	4
2.2.4 RAG 检索增强——src/rag_service.py	5
2.2.5 LLM 多阶段分析——src/api_service.py	5
2.2.6 FastAPI 前端集成——src/app.py	5
2.3 检测结果分析	7
2.4 判断依据的分析	8
3 工作总结	8
3.1 收获与心得	8
3.2 问题与解决	9
3.3 分工与合作	9
4 课程建议	9

1 任务目标

本项目围绕“可解释的谣言检测”任务展开，目标是构建一个能够对英文推文文本进行自动检测并给出判断依据的系统。

系统输入为一段英文文本；输出包括两部分：一是二分类检测结果，其中 0 表示非谣言，1 表示谣言；二是自然语言形式的判断依据，用于解释系统作出该判断的主要原因。

2 系统设计与实现

2.1 实施方案

本项目采用 **BERTweet 小模型分类 + RAG 检索增强 + 大语言模型分析 + 前端展示** 的复合架构。其流程如图 1 所示：

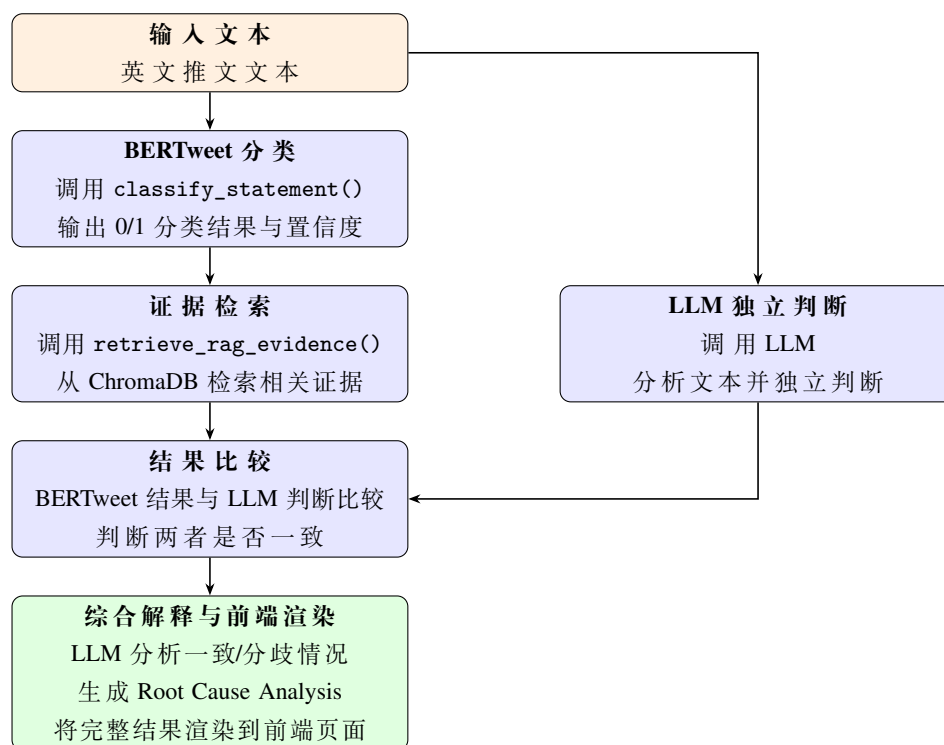


图 1: 系统处理流程图

系统最终通过前端页面展示完整检测结果，包含以下 5 部分：

- 机器学习模型的输出判断及其置信度；
- 大语言模型对于输入文本的独立输出判断；
- 大语言模型对于前两者分析结果的比较及分析；
- 检索增强生成所得到的相关证据摘要。

2.2 核心代码分析

本项目代码位于 `src/` 目录下，按功能划分为数据处理、模型定义与推理、训练流程、RAG 检索、LLM 多阶段分析和前端集成六个模块。以下逐一分析各模块的核心实现。

2.2.1 数据处理模块——src/data.py

数据处理模块负责 CSV 数据的加载、清洗和 PyTorch Dataset 封装。核心函数 `load_datasets()` 接收配置字典，依次读取训练集和验证集 CSV 文件，并委托 `_clean_split()` 完成单表清洗。

`_clean_split()` 的清洗流程分为四步：(1) 对每行文本调用 `normalize_text()` 进行 HTML 解码和空白符合并，确保输入与 BERTweet tokenizer 的预期格式一致；(2) 过滤标签不在 `{0,1}` 内的无效行；(3) 若配置了 `remove_conflicting_texts`，则识别并移除同一文本对应多个不同标签的冲突样本；(4) 若配置了 `drop_duplicate_texts`，则移除文本内容相同的重复行，仅保留首条。清洗完成后返回清洗报告，统计各步骤的移除数量。

`load_datasets()` 在分别清洗训练集和验证集后，额外执行训练集--验证集重叠检测，将验证集中与训练集文本重复的样本删除，防止数据泄漏导致评估失真。

封装后的 `TweetDataset` 类继承 `torch.utils.data.Dataset`，在 `__getitem__()` 中调用 tokenizer 对文本进行编码（padding 至 `max_length`，截断超出部分），返回包含 `input_ids`、`attention_mask` 和 `labels` 的字典，直接适配 Hugging Face `AutoModelForSequenceClassification` 的输入格式。

2.2.2 模型定义与推理——src/model.py、src/predict.py

谣言检测模型基于 Hugging Face 的 `AutoModelForSequenceClassification`，输出层为 2 分类线性头。

`src/model.py` 采用懒加载单例模式对外提供推理接口。模块级变量 `_model`、`_tokenizer`、`_device` 初始为 `None`；首次调用 `classify_statement()` 时触发 `_ensure_model_loaded()`，从训练产出的 `best_model` 目录加载微调后的模型和分词器，并切换至评估模式（`model.eval()`）。

若检查点不存在，自动回退为 `_mock_classify()`，输出随机结果以保证前端测试可用。

核心推理函数 `classify_statement(statement: str) -> dict` 的执行流程为：(1) 对输入文本进行 HTML 解码和空白符合并；(2) 调用 tokenizer 编码为固定长度张量（`max_length=128`，截断填充）；(3) 在 `torch.no_grad()` 上下文中前向传播，对输出的 logits 施加 softmax 得到类别概率分布；(4) 取概率最大的类别作为预测标签（0 或 1），并将对应概率作为置信度返回。

`src/predict.py` 提供了命令行推理入口，支持 `-text`、`-config`、`-checkpoint` 参数，输出包含标签和两类概率的 JSON，便于离线批量测试。

2.2.3 训练流程——src/train.py

训练脚本 `train()` 函数实现了完整的微调流水线，关键设计要点如下：

学习率调度与优化器配置。采用 AdamW 优化器（`lr=1e-5`，`weight_decay=0.01`）配合线性预热衰减调度（`warmup_ratio=0.1`），预热步数占总步数的 10%，在预热阶段学习率从 0 线性增长至目标值，随后线性衰减至 0。

混合精度与梯度累积。支持自动混合精度训练（AMP），在 CUDA 设备上使用 `torch.amp.GradScaler` 和 `autocast` 加速前向传播；通过 `gradient_accumulation_steps` 控制梯度累积步数，等效增大批大小而不增加显存消耗。每步累积后调用 `clip_grad_norm()` 进行梯度裁剪（`max_grad_norm=1.0`），防止梯度爆炸。

早停机制。每个 epoch 结束后在验证集上计算 macro-F1，若连续 `patience=2` 个 epoch 未刷新最佳分数，则终止训练并将最佳检查点保存至 `best_model/` 目录。最佳模型的 tokenizer、权重和评估指标一并持久化，供后续推理模块懒加载使用。

训练曲线绘制。`plot_history()` 函数使用 matplotlib 在每个 epoch 后更新四幅曲线图（训练/验证 loss、验证 accuracy、验证 macro-F1 和梯度范数），输出至 `outputs/bertweet/plots/` 目录，便于训练过程中实时监控。

2.2.4 RAG 检索增强——src/rag_service.py

RAG 模块为 LLM 分析提供辅助证据, 不参与 BERTweet 的分类决策。函数 `retrieve_rag_evidence(statement, top_k=3)` 动态导入 `RAG/chroma_retriever.py` 中的 `ChromaRetriever` 类, 调用其 `retrieve()` 方法从 ChromaDB 向量数据库中检索与输入文本最相似的 top-k 条样本, 返回包含来源、标签、距离和文本内容的证据列表。

模块采用防御式设计: 所有 RAG 相关逻辑包裹在 `try/except` 块中, 若 ChromaDB 数据库不存在、依赖未安装或检索过程出错, 函数静默返回空列表, 确保主检测流程不受 RAG 故障影响。该设计体现了“RAG 为可选的解释增强模块”的系统定位。

2.2.5 LLM 多阶段分析——src/api_service.py

LLM 分析模块是整个系统可解释性的核心, 实现了三阶段提示工程管道:

第一阶段: 独立判断 (`judge_statement()`)。LLM 在未看到 BERTweet 分类结果的情况下, 结合输入文本和 RAG 检索证据进行独立分析。系统提示词要求 LLM 从可验证性、信源可信度、逻辑连贯性、情绪化语言和文本具体性五个维度评估, 输出 JSON 格式的二元判断、标签、推理过程和支撑线索。温度参数设为 0.3 以减小随机性, RAG 证据在用户提示词中以格式化文本呈现, 并强调“证据仅供参, 不应盲目信任”。

第二阶段: 结果比较 (`compare_results()`)。将 BERTweet 的分类结果 (标签 + 置信度) 与 LLM 第一阶段结论并列呈现, 要求 LLM 判断两者是否一致并生成比较摘要。该阶段温度同样设为 0.3, 输出经过 JSON 解析提取。

第三阶段: 根因分析 (`analyze_root_cause()`)。根据第二阶段的一致性判断, 分支选择不同的提示词模板——一致时提示 LLM 综合双方证据生成统一解释 (`STAGE3_CONVERGE_PROMPT`), 分歧时提示 LLM 分析双方结论相悖的原因并评估哪一方更可信 (`STAGE3_DIVERGE_PROMPT`)。

三阶段设计实现了“独立判断—交叉验证—根因追溯”的递进分析模式, 输出层层递进, 兼顾检测结果的可解释性和分歧诊断的透明度。

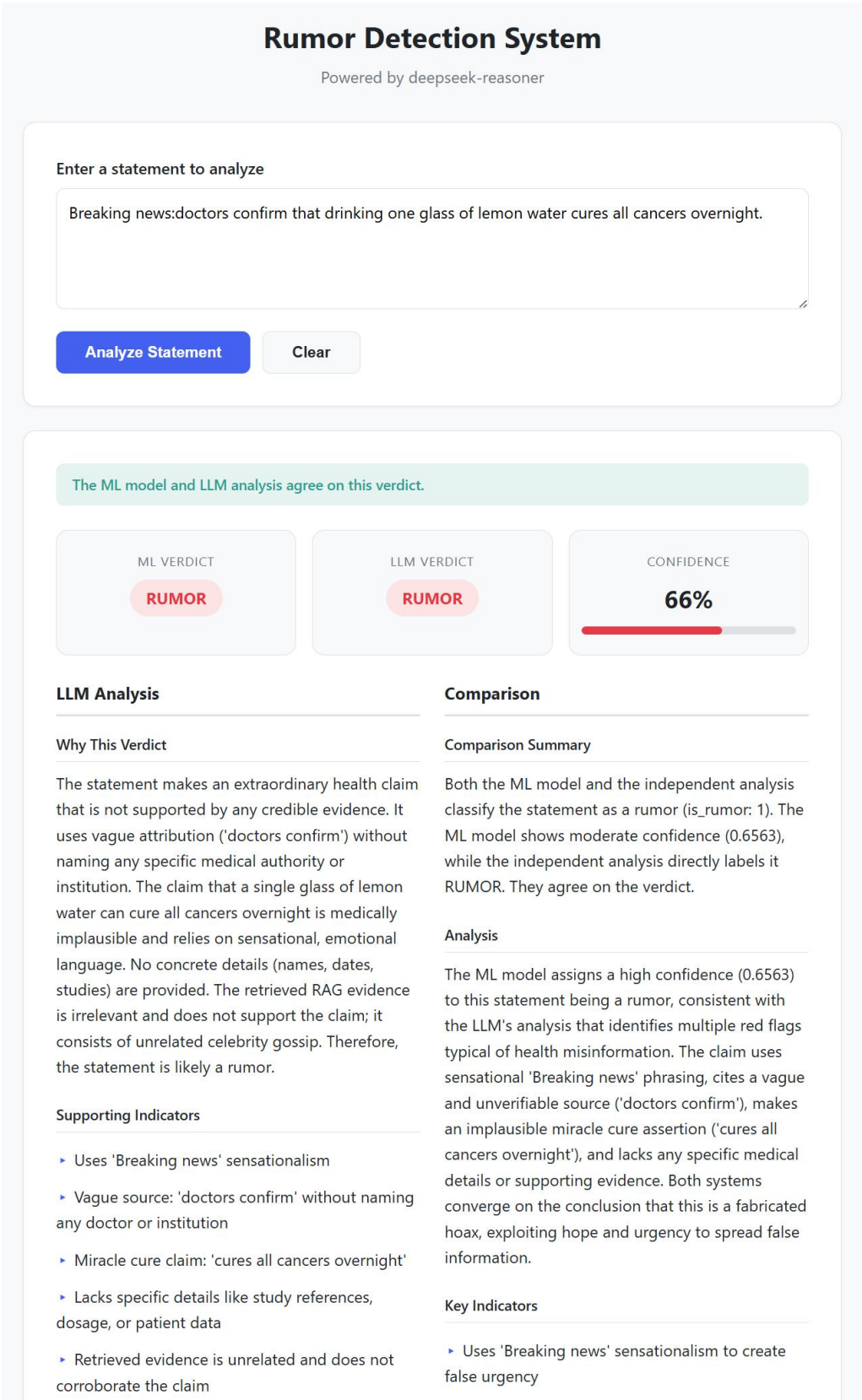
JSON 解析的鲁棒性设计。辅助函数 `_parse_json()` 采用三级回退策略: 优先直接解析裸 JSON; 若失败则用正则匹配 `"'json ... '"` 代码块; 若仍未成功, 则用正则提取首个花括号包围的片段。三级回退确保即使 LLM 输出格式出现波动 (如额外 markdown 包裹), 系统仍能提取结构化结果。

2.2.6 FastAPI 前端集成——src/app.py

前端使用 FastAPI 框架构建, 核心为单一 POST 端点 `/analyze`。`create_app()` 工厂函数创建 FastAPI 实例并注册 Jinja2 模板引擎。

`analyze()` 处理函数按以下顺序编排五个步骤: (1) `classify_statement()` 进行 BERTweet 分类; (2) `retrieve_rag_evidence()` 检索 RAG 证据 (失败则回退为空列表); (3) `judge_statement()` 执行 LLM 第一阶段独立判断; (4) `compare_results()` 比较 ML 与 LLM 结果; (5) `analyze_root_cause()` 生成根因分析。每个步骤封装独立的异常处理, 任一步骤失败均向用户返回具体错误信息, 避免系统整体崩溃。

最终返回的模板上下文包含完整结果集: ML Verdict、LLM Verdict、Confidence、LLM Reasoning、Comparison Summary、Root Cause Analysis、Key Indicators 和 RAG Evidence, 所有数据渲染至 `index.html` 展示, 如图2所示。



2.3 检测结果分析

本项目使用 `train.csv` 微调 BERTweet 模型，并在 `val.csv` 上验证。数据清洗后，训练集共 2832 条样本（非谣言 1596 条，谣言 1236 条），验证集共 400 条样本（非谣言 225 条，谣言 175 条），类别分布基本均衡，不存在严重的长尾偏差。训练配置方面，采用学习率 1×10^{-5} 、批大小 32，最大训练 16 个 epoch，以验证集 macro-F1 作为早停指标（`patience=2`），即连续两个 epoch 未提升则终止训练。

实验结果显示，模型在第 5 个 epoch 取得最佳验证集表现：accuracy 为 0.8700，macro-F1 为 0.8688，验证集 loss 为 0.3696。早停机制在第 7 个 epoch 后触发，系统最终保存第 5 轮 macro-F1 最优的模型权重作为最终检测模型。

图3、图4与图5分别展示了验证集 accuracy、macro-F1 以及训练/验证 loss 随 epoch 的变化趋势。从 accuracy 与 macro-F1 曲线可以观察到，两项指标在前 5 轮持续上升，第 5 轮达到峰值后出现回落，第 6–7 轮均稳定在 0.8625 附近，不再继续改善。从 loss 曲线可以更清晰地看出过拟合现象：训练集 loss 从 0.6735 单调递减至 0.1256，表明模型对训练数据的拟合程度持续加深；而验证集 loss 在第 5 轮降至 0.3696 后开始反弹（第 6 轮 0.4160，第 7 轮 0.4208），训练 loss 与验证 loss 之间的差距从第 5 轮的 0.161 扩大至第 7 轮的 0.295，呈现典型的过拟合特征——模型在训练集上继续优化，但泛化能力反而退化。

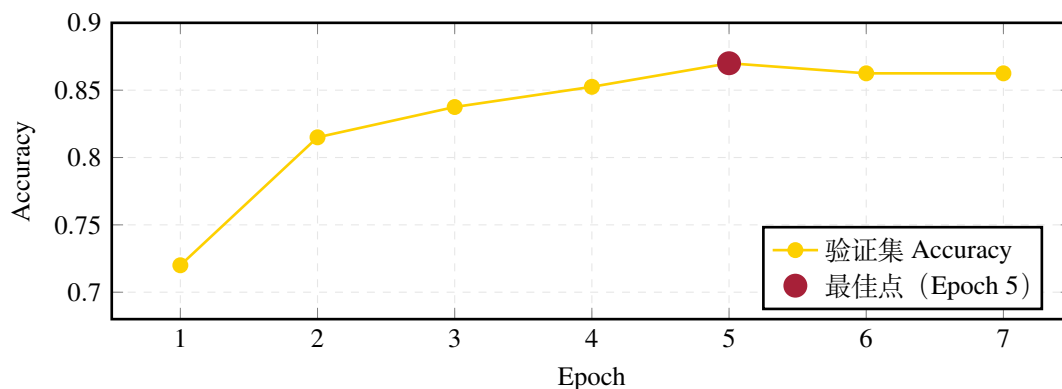


图 3: 验证集 Accuracy 随训练轮次变化曲线

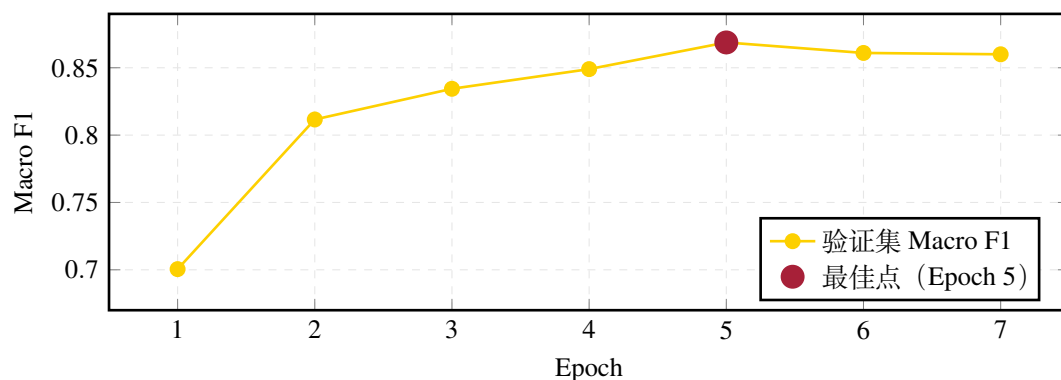


图 4: 验证集 Macro F1 随训练轮次变化曲线

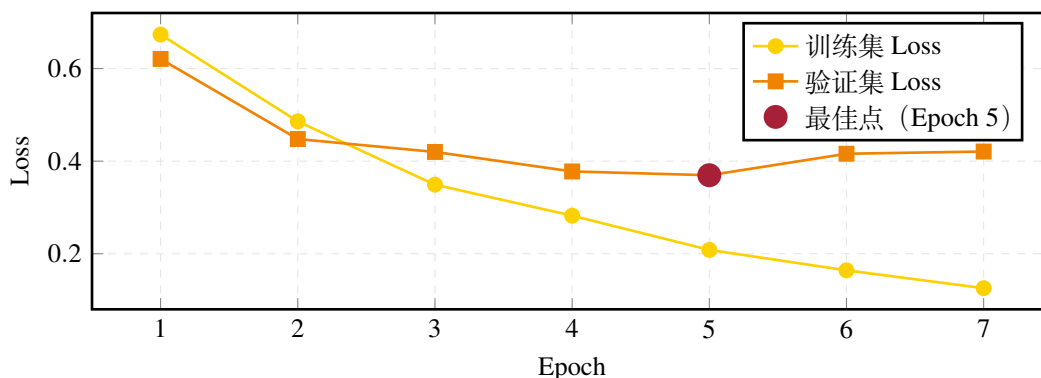


图 5: 训练集与验证集 Loss 随训练轮次变化曲线

最佳模型在验证集上的混淆矩阵如表 1 所示。模型对非谣言类 (label 0) 的精确率为 0.9061, 召回率为 0.8578, F1 得分为 0.8813; 对谣言类 (label 1) 的精确率为 0.8289, 召回率为 0.8857, F1 得分为 0.8564。谣言类的召回率 (0.8857) 略高于精确率 (0.8289), 表明模型倾向于将边界样本判为谣言, 检出能力较强但存在一定误报——32 例非谣言被误判为谣言 (假阳性), 20 例谣言被漏判为非谣言 (假阴性)。从应用角度看, 谣言检测任务中漏判 (假阴性) 的危害通常大于误报, 因此模型较高的谣言召回率是符合实际需求的特征。

		预测标签	
		0 (非谣言)	1 (谣言)
真实标签	0 (非谣言)	193	32
	1 (谣言)	20	155

表 1: 最佳模型 (Epoch 5) 验证集混淆矩阵

综上, 微调后的 BERTweet 模型在较小规模数据集上取得了较为鲁棒的分类性能, accuracy 与 macro-F1 均达到 0.87 左右, 早停策略有效抑制了过拟合, 模型在谣言检出率和非谣言精确率之间保持了合理的平衡。

2.4 判断依据的分析

本项目的判断依据主要由大语言模型生成, 结合 RAG 检索证据提供辅助参考。LLM 会从可验证性、信源可信度、逻辑连贯性、情绪化语言和文本具体性等角度生成解释。

当 ML 与 LLM 判断一致时, 系统会生成统一解释; 当两者判断不一致时, 系统会提示分歧并分析原因。页面下方展示 RAG Retrieved Evidence, 包括证据来源、标签、距离和文本内容, 增加判断过程的透明度。实验中也发现, RAG 检索结果会受到数据库内容影响, 因此本项目将 RAG 定位为解释增强模块, 而不是最终分类依据。

3 工作总结

3.1 收获与心得

通过本项目, 我们完整实践了一个 AI 应用系统的设计与实现过程, 包括数据处理、预训练模型微调、模型评估、RAG 检索增强、大语言模型 API 调用和前端展示。项目加深了我们对 Transformer 文本分类模型、RAG 检索机制和可解释 AI 系统设计的理解。

3.2 问题与解决

项目过程中主要遇到三个问题。

1. 数据集规模较小，BERTweet 型训练过多轮后容易过拟合。对此，我们采用验证集 macro-F1 作为早停指标，并保存最佳模型。
2. RAG 数据库较大且检索相关性存在波动，因此数据库通过 GitHub Release 提供，系统将 RAG 作为可选解释增强模块接入，保证主系统稳定运行。
3. LLM 解释可能与小模型分类结果不一致。对此，我们设计了多阶段分析流程，将小模型判断、LLM 判断和分歧分析分开展示。

3.3 分工与合作

表2总结了我們小组成员的具体分工和贡献。

成员	分工	贡献
陈忠淳	项目统筹、小模型训练与评估	GitHub 仓库管理、BERTweet 微调与评估
沈润泽	RAG 模块开发	外部数据整理、ChromaDB 向量数据库构建、RAG 独立运行测试
戴铭辰	前端与大模型调用	FastAPI/Jinja2 前端搭建与 API 调用、LLM 三阶段分析流程设计
谢紫浩	报告与文档整理	实验报告撰写、README 更新、RAG 接入和最终运行检测

表 2: 小组成员分工与贡献

4 课程建议

建议后续课程中可以提供更多关于模型部署、API 调用、RAG 接入和可解释性评估的知识讲解和应用示例，以帮助学生更好地完成从模型训练到系统实现的完整流程。增加可获取数据集的途径等，方便学生进一步探索。